Name:- Raghav Anil Bhise
Roll No. :- 391061
PRN:- 22311792

1. Title

Implementation of Bag of Words (Count Occurrence) Model for Text Representation

---

2. Aim

To implement the **Bag of Words (BoW) model using Count Occurrence** technique for converting textual data into numerical form for Natural Language Processing applications.

---

3. Objectives

- To understand the concept of the **Bag of Words model**
- To preprocess text data for NLP tasks
- To generate a **word count matrix** from text documents
- To represent documents numerically using **Count Vectorization**

---

4. Theory

4.1 Natural Language Processing

Natural Language Processing (NLP) enables machines to analyze and interpret human language. Since machine learning models cannot process text directly, text must be transformed into numerical features.

4.2 Bag of Words (BoW) Model

The Bag of Words model represents text by counting the frequency of words in a document, **ignoring grammar and word order**.

Each document is represented as a vector where:

- Columns represent unique words (vocabulary)
- Values represent the **number of times each word appears**

4.3 Count Occurrence Method

In Count Occurrence:

- Every word's frequency is counted
- Common words appear with higher values
- No weighting or importance reduction is applied

This method is simple and effective for basic text classification tasks.

4.4 Advantages and Limitations

Advantages

- Easy to implement
- Computationally efficient
- Useful for basic NLP models

Limitations

- Ignores word order and semantics
- Common words may dominate the representation

---

5. Program Codes

Bag of Words Algorithm Implementation

```
[ ]
''' vectorize() function takes list of words in a sentence as input
    and returns a vector of size of filtered_vocab.It puts 0 if the
    word is not present in tokens and count of token if present.'''

def vectorize(tokens):
    vector=[]
    for w in filtered_vocab:
        vector.append(tokens.count(w))
    return vector
```

```
[ ]
'''unique() functions returns a list in which the order remains
    same and no item repeats.Using the set() function does not
    preserve the original ordering,so i didnt use that instead'''

def unique(sequence):
    seen = set()
    return [x for x in sequence if not (x in seen or seen.add(x))]
#create a list of stopwords.You can import stopwords from nltk too
stopwords=["to","was","a"]
#list of special characters.You can use regular expressions too
special_char=[",",":"," ",";",".","?"]
```

```
[ ]
string1 = "Data science helps find meaningful insights"
string2 = "Data science finds insights"
string3 = "Science reveals insights"

#converting strings to lower case
string1=string1.lower()
string2=string2.lower()
string3=string3.lower()

#split the sentences into tokens
tokens1=string1.split()
tokens2=string2.split()
tokens3=string3.split()

print(tokens1)
print(tokens2)
print(tokens3)
```

```
['data', 'science', 'helps', 'find', 'meaningful', 'insights']
['data', 'science', 'finds', 'insights']
['science', 'reveals', 'insights']
```

```
[ ]
#create a vocabulary list
vocab=unique(tokens1+tokens2+tokens3)
print(vocab)


#filter the vocabulary list
filtered_vocab=[]

for w in vocab:
    if w not in stopwords and w not in special_char:
        filtered_vocab.append(w)
print("Final filtered vocabulary: ", filtered_vocab)



#convert sentences into vectords
vector1=vectorize(tokens1)
print("Sentence 1 vector :",vector1)

vector2=vectorize(tokens2)
print("Sentence 2 vector :",vector2)

vector3=vectorize(tokens3)
print("Sentence 3 vector :",vector3)
```

```
['data', 'science', 'helps', 'find', 'meaningful', 'insights', 'finds', 'reveals']
Final filtered vocabulary:  ['data', 'science', 'helps', 'find', 'meaningful', 'insights', 'finds', 'reveals']
Sentence 1 vector : [1, 1, 1, 1, 1, 1, 0, 0]
Sentence 2 vector : [1, 1, 0, 0, 0, 1, 1, 0]
Sentence 3 vector : [0, 1, 0, 0, 0, 1, 0, 1]
```

Creating Bag of Words using sklearn library

```python
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

string1 = "Data science analyzes data patterns"
string2 = "Data science finds data patterns"
string3 = "Science analyzes patterns"
string4 = "Data contains patterns"

doc = string1+string2+string3+string4

CountVec = CountVectorizer(ngram_range=(1,1))
#transform
Count_data = CountVec.fit_transform([string1,string2,string3,string4])

#create dataframe
# Use get_feature_names_out() instead of get_feature_names()
cv_dataframe=pd.DataFrame(Count_data.toarray(),columns=CountVec.get_feature_names_out())
print(cv_dataframe)
```

```
   analyzes  contains  data  finds  patterns  science
0         1         0     2      0         1        1
1         0         0     2      1         1        1
2         1         0     0      0         1        1
3         0         1     1      0         1        0
```

Note that the CountVectorize sorts the vocabulary alphabetically before generating vectors.

Count Occurrence

```python
count_vec = CountVectorizer()
count_occurs = count_vec.fit_transform([doc])
count_occur_df = pd.DataFrame((count, word) for word, count in zip(count_occurs.toarray().tolist()[0], count_vec.get_feature_names_out()))
count_occur_df.columns = ['Word', 'Count']
count_occur_df.sort_values('Count', ascending=False, inplace=True)
count_occur_df.head()
```

	Word	Count
2	data	3
0	analyzes	2
7	science	2
5	patternsdata	2
3	finds	1

Normalized Count Occurrence

```python
from sklearn.feature_extraction.text import TfidfVectorizer
norm_count_vec = TfidfVectorizer(use_idf=False, norm='l2')
norm_count_occurs = norm_count_vec.fit_transform([doc])
norm_count_occur_df = pd.DataFrame((count, word) for word, count in zip(
    norm_count_occurs.toarray().tolist()[0], norm_count_vec.get_feature_names_out()))
norm_count_occur_df.columns = ['Word', 'Count']
norm_count_occur_df.sort_values('Count', ascending=False, inplace=True)
norm_count_occur_df.head()
```

	Word	Count
2	data	0.6
0	analyzes	0.4
7	science	0.4
5	patternsdata	0.4
3	finds	0.2

---

6. Result and Outcomes

Result

- Text documents were successfully converted into a **Bag of Words count matrix**.
- Each row represents a document and each column represents a word.

- The matrix contains **word frequency values**.

Outcomes

- Learned how CountVectorizer works
- Understood text preprocessing and feature extraction
- Successfully implemented Bag of Words model
- Prepared data for further NLP tasks such as classification

---

7. Conclusion

The Bag of Words (Count Occurrence) model is a fundamental technique in Natural Language Processing for converting text into numerical form. Although it does not capture semantic meaning or word order, it is effective for simple NLP tasks. This experiment helped in understanding the basic mechanism of text vectorization using word frequency counts.